ISOP Architecture Diagrams

TP Owned by **Tejaswini Panjakadi** · Software version 0.3.0 **DRAFT**

Two views of the same system. **Figure 1** explains what ISOP does at a glance, for any audience. **Figure 2** shows every module with its technology, interfaces and security controls, for technical review. Both diagrams are standalone SVG files in `docs/diagrams/` and can be inserted into Word or PowerPoint without losing sharpness.

Contents

1. High-level architecture (Figure 1) and component overview
2. Technical architecture (Figure 2) and tier summary

All fraud signals in ISOP are simulated. The AI planner advises which checks to run; only the deterministic policy engine decides ALLOW, REVIEW or REJECT.

1. High-level architecture

A registration is investigated entirely on one laptop. The AI planner only advises which extra checks to run; the deterministic policy engine alone decides ALLOW, REVIEW or REJECT, and every step is recorded and streamed to the dashboard.

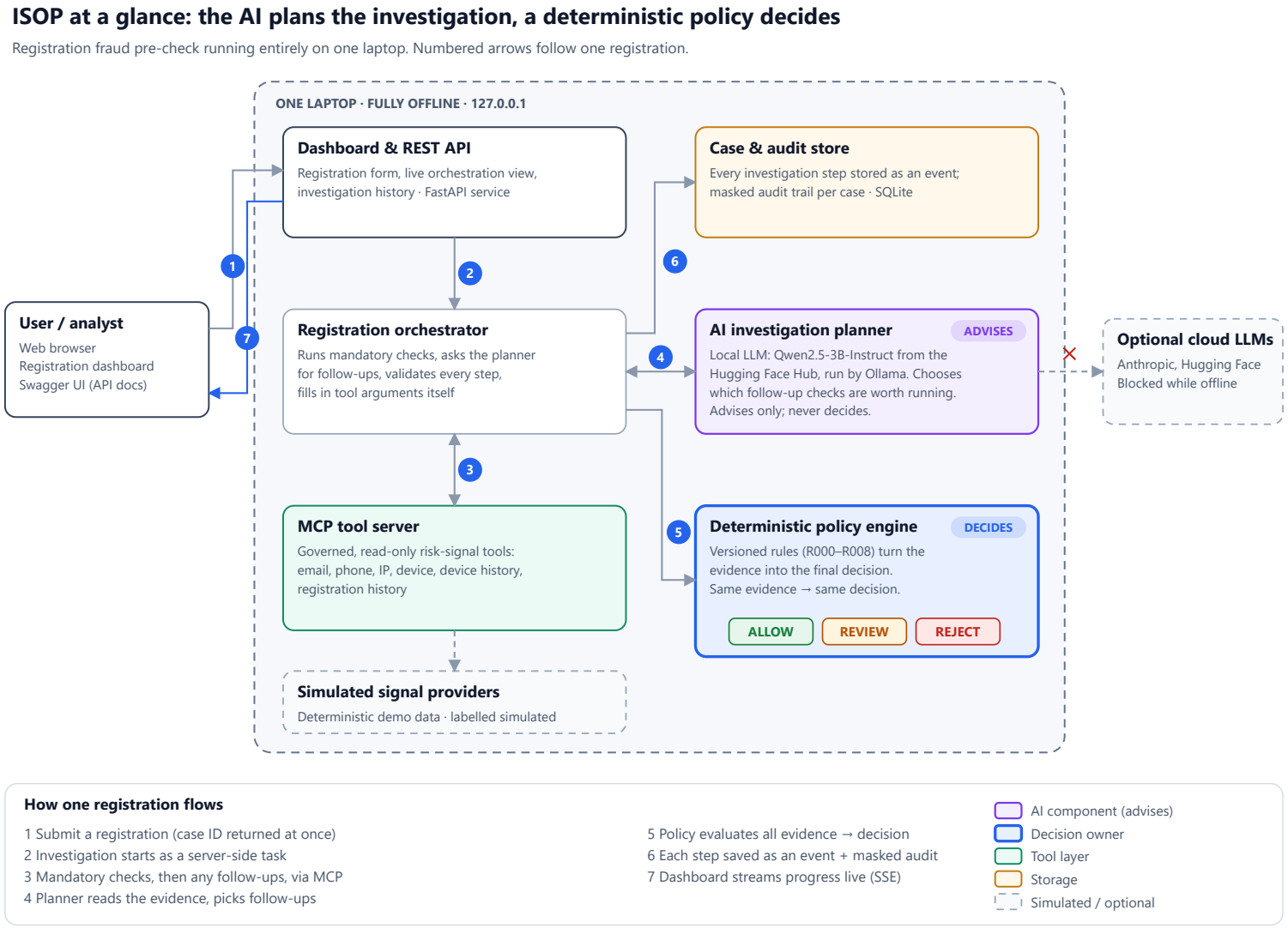


Figure 1 · High-level architecture and the path of one registration

Component	Role	Technology
Dashboard & REST API	Registration form, live orchestration view, history; API for the dashboard and other clients	FastAPI, vanilla JavaScript
Registration orchestrator	Runs mandatory checks, consults the planner, validates its choices, calls tools	Python asyncio
AI investigation planner	Chooses which follow-up checks are worth running (advises only)	Qwen2.5-3B-Instruct from the Hugging Face Hub, run locally by Ollama
MCP tool server	Governed, read-only risk-signal tools, backed by simulated providers	Model Context Protocol (stdio)
Deterministic policy engine	Turns the evidence into ALLOW / REVIEW / REJECT with reason codes	Versioned rules in JSON
Case & audit store	Every step as an event, plus a masked audit trail per case	SQLite

2. Technical architecture

Five tiers, top to bottom in the order a request travels. Purple components plan, the blue component decides, green components gather evidence through MCP, and amber components persist data. The dashed provider box is optional and refused while `ISOP_OFFLINE=true`.

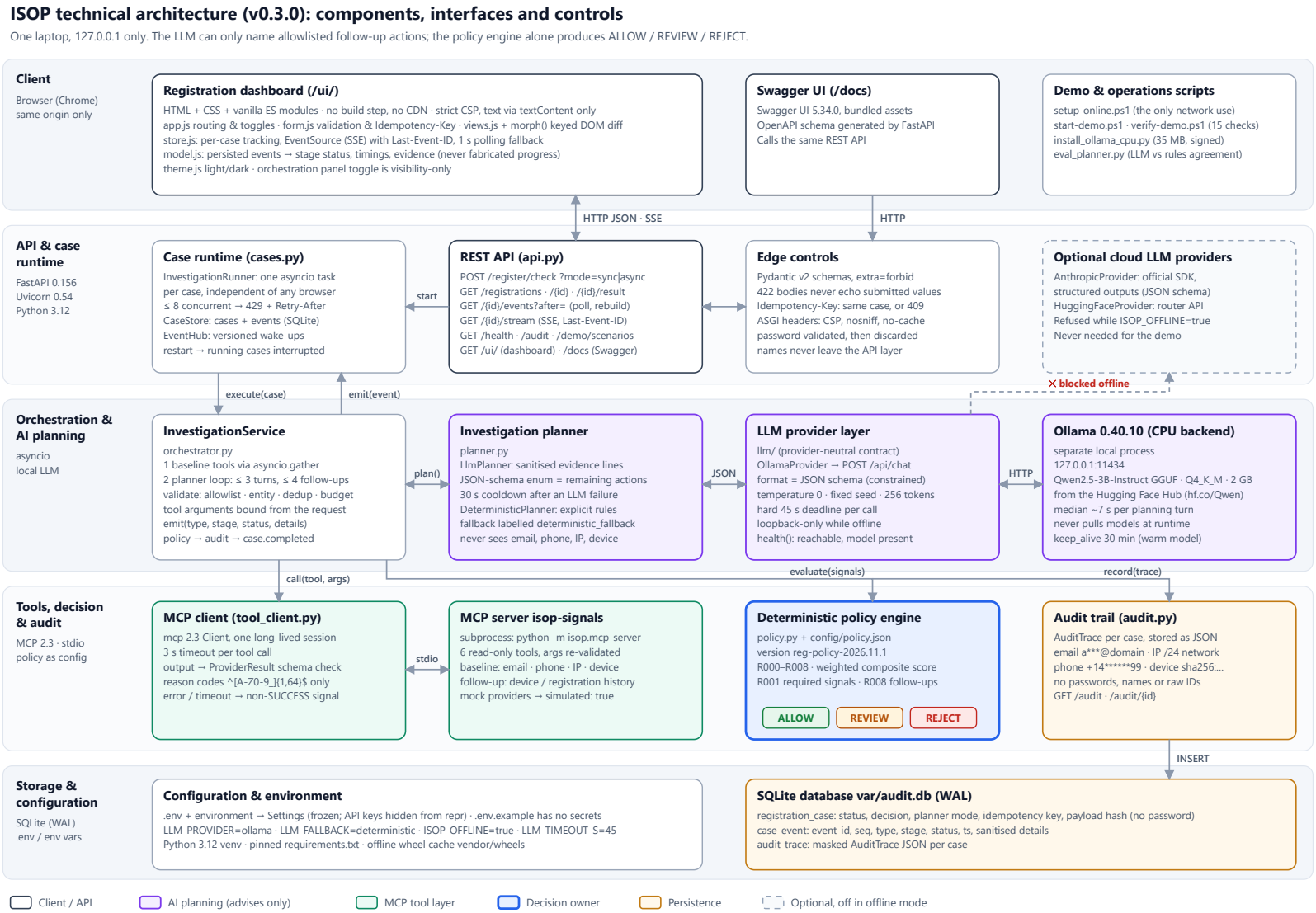


Figure 2 · Technical architecture with interfaces and controls

Tier	Key components	Interfaces	Controls
Client	Dashboard (`/ui/`), Swagger UI (`/docs`), PowerShell demo scripts	HTTP JSON, Server-Sent Events	Strict CSP, text-only rendering, no CDN
API & case runtime	`api.py` , `cases.py` (runner, case store, event hub)	`POST /register/check` , `/registrations/*` , `/stream`	Schema validation, Idempotency-Key, 8-case limit, no echoed input
Orchestration & AI planning	InvestigationService, LlmPlanner / DeterministicPlanner, provider layer, Ollama	`plan()` , `POST /api/chat` on 127.0.0.1	Allowlisted actions, schema-constrained output, 45 s deadline, labelled fallback
Tools, decision & audit	MCP client and server, policy engine, audit trail	MCP over stdio, `evaluate()` , `record()`	Argument re-validation, fail closed, rules R000–R008, masking
Storage & configuration	SQLite `var/audit.db` , `.env` / Settings	SQL (WAL)	No passwords stored, secrets hidden from logs

---

Diagrams generated from the ISOP v0.3.0 code base by `docs/diagrams/generate.py` , 5 November 2026.